

WES 237A: Introduction to Embedded System Design (Winter 2026)

Lab 5: Inter-Integrated Circuit (I2C) Communication

Due: 3/1/2026 11:59pm

In order to report and reflect on your WES 237A labs, please complete this Post-Lab report by the end of the weekend by submitting the following 2 parts:

- Upload your lab 5 report composed by a single PDF that includes your in-lab answers to the bolded questions in the Google Doc Lab and your Jupyter Notebook code. You could either scan your written copy, or simply type your answer in this Google Doc. **However, please make sure your responses are readable.**
- Answer two short essay-like questions on your Lab experience.

All responses should be submitted to Canvas. Please also be sure to push your code to your git repo as well.

- Connect the PMOD_AD2 peripheral to PMODA.
- Download the [iic_example.ipynb](#)
- Go through the notebook and answer the following questions. The following resources may be helpful
 - https://pynq.readthedocs.io/en/v2.6.1/pynq_libraries/pynqmb_reference.html
 - https://www.analog.com/media/en/technical-documentation/data-sheets/AD7991_7995_7999.pdf
 - https://pynq.readthedocs.io/en/v2.1/pynq_package/pynq.lib/pynq.lib.pmod.html#pynq-lib-pmod

- **What command opens a new I2C device in the MicroblazeLibrary? What are the two parameters to this command?**

```
i2c_open(3, 2);
```

```
i2c i2c_open(int sda, int scl)
```

Open an I2C device attached to an I/O switch configured to use the specified pins. Calling this function will disconnect any previously assigned pins and return them to a high-impedance state. – api docs

- **What does 0x28 refer to in the following line?**

- `device.write(0x28, buf, 1)`

```
Issue a write command to the specified slave memory address hex 28 or decimal value 40
```

- **Why do we write and then read when using the Microblaze Library?**

```
We write first to the I2C device what register we want to talk to then we read to retrieve the data it returns in response to that command.
```

- **What does this code snippet mean?** `return ((buf[0] & 0x0F) << 8) | buf[1]`

Takes the lower 4 bits of the first byte, shift them left 8 positions to become the upper part of a 12 bit then with the second byte uses a bitwise OR to return the full 12 bit ADC value

- **What is the difference between writing to the device when using the Microblaze Library and directly on the Microblaze?**

When you write using the MicroBlaze Library in Python, you are calling a high level API that sends your request from the cpu side to a MicroBlaze firmware driver, which then performs the actual I2C write on the Pmod pins, but when you call directly on the MicroBlaze you are running the code on the MicroBlaze itself, calling the I2C driver functions directly, with less software overhead and more direct control over timing and low level behavior.

Using PYNQ library for PMOD_ADC

This just uses the built in Pmod_ADC library to read the value on the PMOD_AD2 peripheral.

```
In [1]: from pynq.overlays.base import BaseOverlay
        from pynq.lib import Pmod_ADC
        base = BaseOverlay("base.bit")
```

```
In [2]: adc = Pmod_ADC(base.PMODA)
```

Read the raw value and the 12 bit values from channel 1.

Refer to docs:

https://pynq.readthedocs.io/en/v2.1/pynq_package/pynq.lib/pynq.lib.pmod.html#pynq-lib-pmod

```
In [19]: adc.read_raw(ch1=1, ch2=0, ch3=0)
```

```
Out[19]: [4095]
```

```
In [20]: adc.read(ch1=1, ch2=0, ch3=0)
```

```
Out[20]: [1.9995]
```

Using MicroblazeLibrary

Here we're going down a level and using the microblaze library to write I2C commands directly to the PMOD_AD2 peripheral

Use the documentation on the PMOD_AD2 to answer lab questions

```
In [30]: from pynq.overlays.base import BaseOverlay
        from pynq.lib import MicroblazeLibrary
        base = BaseOverlay("base.bit")
```

```
In [31]: liba = MicroblazeLibrary(base.PMODA, ['i2c'])
```

```
In [25]: dir(liba) # list the available commands for the liba object
```

```
Out[25]: ['__class__',
          '__delattr__',
          '__dict__',
          '__dir__',
          '__doc__',
          '__eq__',
          '__format__',
          '__ge__',
          '__getattr__',
          '__gt__',
          '__hash__',
          '__init__',
          '__init_subclass__',
          '__le__',
          '__lt__',
          '__module__',
          '__ne__',
          '__new__',
          '__reduce__',
          '__reduce_ex__',
          '__repr__',
          '__setattr__',
          '__sizeof__',
          '__str__',
          '__subclasshook__',
          '__weakref__',
          '_build_constants',
          '_build_functions',
          '_mb',
          '_populate_typedefs',
          '_rpc_stream',
          'active_functions',
          'i2c_close',
          'i2c_get_num_devices',
          'i2c_open',
          'i2c_open_device',
          'i2c_read',
          'i2c_write',
          'release',
          'reset',
          'visitor']
```

In the cell below, open a new i2c device. Check the resources for the i2c_open parameters

```
In [34]: device = liba.i2c_open(3,2) # TODO open a device
```

```
In [35]: dir(device) # list the commands for the device class
```

```
Out[35]: ['__class__',
          '__delattr__',
          '__dict__',
          '__dir__',
          '__doc__',
          '__eq__',
          '__format__',
          '__ge__',
          '__getattr__',
          '__gt__',
          '__hash__',
          '__index__',
          '__init__',
          '__init_subclass__',
          '__int__',
          '__le__',
          '__lt__',
          '__module__',
          '__ne__',
          '__new__',
          '__reduce__',
          '__reduce_ex__',
          '__repr__',
          '__setattr__',
          '__sizeof__',
          '__str__',
          '__subclasshook__',
          '__weakref__',
          '_call_func',
          '_file',
          '_val',
          'close',
          'read',
          'write']
```

Below we write a command to the I2C channel and then read from the I2C channel. Change the buf[0] value to select different channels. See the AD spec sheet Configuration Register. https://www.analog.com/media/en/technical-documentation/data-sheets/AD7991_7995_7999.pdf

Changing the number of channels to read from will require a 2 byte read for each channel!

```
In [66]: buf = bytearray(2)
          buf[0] = int('00000000', 2)
          device.write(0x28, buf, 1)
          device.read(0x28, buf, 2)
          print(format(int(((buf[0] << 8) | buf[1])), '#018b'))
```

```
0b00000000100110100
```

Compare the binary output given by ((buf[0]<<8) | buf[1]) to the AD7991 spec sheet. You can select the data only using the following command

```
In [67]: result_12bit = (((buf[0] & 0x0F) << 8) | buf[1])
         print(result_12bit)
```

308

Using MicroBlaze

```
In [68]: base = BaseOverlay("base.bit")
```

```
In [70]: %%microblaze base.PMODA
```

```
#include "i2c.h"
```

```
int read_adc(){
    i2c i2c_device = i2c_open(3, 2);
    unsigned char buf[2];
    buf[0] = 0;
    i2c_write(i2c_device, 0x28, buf, 1);
    i2c_read(i2c_device, 0x28, buf, 2);
    return ((buf[0] & 0x0F) << 8) | buf[1];
}
```

```
In [71]: read_adc()
```

Out[71]: 311

```
In [ ]:
```